

architecture

Architecture

This document describes the high-level architecture of pkg/userflow, covering its core design patterns, component layers, and how they integrate with the broader challenge framework.

Layered Design

The package is organized into four layers:

+-----+		
	Challenge Templates	
	(BrowserFlowChallenge, APIFlowChallenge, etc.)	
+-----+		
	Flow Definitions	
	(BrowserFlow, APIFlow, MobileFlow, IPCCommand)	
+-----+		
	Platform Adapters	
	Interfaces: BrowserAdapter, APIAdapter, etc.	
	Implementations: PlaywrightCLI, ADBCLI, etc.	
+-----+		
	Infrastructure	
	TestEnvironment, Plugin, Evaluators, ResultParsers	
+-----+		

Layer 1: Infrastructure

The bottom layer provides supporting services:

- **UserFlowPlugin** -- Registers 12 assertion evaluators with the challenge framework's assertion engine. It implements the plugin.Plugin interface and is initialized during application bootstrap.
- **TestEnvironment** -- Manages containerized test infrastructure using the Containers module (runtime detection, compose orchestration, health checking, service registry, event bus).

- **Result parsers** -- Convert `TestResult` and `BuildResult` into maps of values and metrics suitable for assertion evaluation (`ParseTestResultToValues`, `ParseBuildResultToMetrics`, etc.).
- **Functional options** -- `ChallengeOption` values configure challenges with settings like `WithContainerized`, `WithProjectRoot`, and `WithRuntimeName`.

Layer 2: Platform Adapters

Six adapter interfaces abstract platform-specific operations:

Interface	Purpose	Built-in Implementation
<code>BrowserAdapter</code>	Browser UI automation	<code>PlaywrightCLIAdapter</code>
<code>MobileAdapter</code>	Mobile device testing	<code>ADBCLIAdapter</code>
<code>DesktopAdapter</code>	Desktop app testing	<code>TauriCLIAdapter</code>
<code>APIAdapter</code>	REST API and Web-Socket	<code>HTTPAPIAdapter</code>
<code>BuildAdapter</code>	Build, test, lint	<code>GradleCLIAdapter</code> , <code>NPMCLIAdapter</code> , <code>GoCLIAdapter</code> , <code>CargoCLIAdapter</code>
<code>ProcessAdapter</code>	Process life-cycle	<code>ProcessCLIAdapter</code>

Each interface follows the same conventions:

- Every method takes `context.Context` as its first parameter.
- An `Available(ctx) bool` method checks whether the underlying tool is installed and usable.
- A `Close(ctx) error` method (where applicable) releases resources.

Layer 3: Flow Definitions

Flow types define declarative step sequences that challenge templates execute:

- **BrowserFlow** -- Named sequence of `BrowserStep` values (`navigate`, `click`, `fill`, `select`, `wait`, `assert_visible`, `assert_text`, `assert_url`, `screenshot`, `evaluate_js`). Each step may carry `StepAssertion` values.
- **APIFlow** -- Named sequence of `APIStep` values (`GET`, `POST`, `PUT`, `DELETE`) with optional credentials, variable extraction (`ExtractTo`), and per-step assertions.

- **MobileFlow** -- Named sequence of MobileStep values (launch, tap, send_keys, press_key, screenshot, wait, stop, assert_running).
- **IPCCommand** -- Single IPC command definition for desktop backend invocation, with expected result and assertions.

All flow types are JSON-serializable, enabling definition in configuration files.

Layer 4: Challenge Templates

Challenge templates are concrete challenge.Challenge implementations that compose an adapter with a flow definition. They embed challenge.BaseChallenge to inherit the challenge lifecycle (ID, dependencies, progress reporting, result creation).

There are 13 challenge template types organized by platform:

- **Environment:** EnvironmentSetupChallenge, EnvironmentTeardownChallenge
- **API:** APIHealthChallenge, APIFlowChallenge
- **Browser:** BrowserFlowChallenge
- **Build:** BuildChallenge, UnitTestChallenge, LintChallenge
- **Mobile:** MobileLaunchChallenge, MobileFlowChallenge, InstrumentedTestChallenge
- **Desktop:** DesktopLaunchChallenge, DesktopFlowChallenge, DesktopIPCChallenge

Design Patterns

Adapter Pattern

The central pattern. Each platform adapter interface defines a contract that can be implemented by different toolchains. This allows challenges to be written against the interface while the concrete tooling (Playwright, ADB, Gradle, etc.) can be swapped out.

```
BrowserFlowChallenge --uses--> BrowserAdapter (interface)
                               |
                               PlaywrightCLIAdapter (concrete)
```

Template Method

Challenge templates use the Template Method pattern via challenge.BaseChallenge. The base provides the lifecycle (Configure, Validate, Cleanup) and utility methods (ReportProgress, CreateResult). Each template overrides Execute() with its platform-specific logic.

Strategy Pattern

The evaluator registry uses the Strategy pattern. Each evaluator function implements the `assertion.Evaluator` signature and is registered by name. The assertion engine selects the correct evaluator at runtime based on the assertion type string.

Functional Options

Configuration uses the functional options pattern via `ChallengeOption` and `TestEnvironmentOption`:

```
env, err := userflow.NewTestEnvironment(  
    userflow.WithComposeFile("compose.test.yml"),  
    userflow.WithProjectName("my-test"),  
    userflow.WithPlatformGroups(groups),  
)
```

Observer (Event Bus)

The `TestEnvironment` integrates the Containers module's `EventBus` for container lifecycle events. This enables reactive monitoring of container state changes during test execution.

Data Flow

A typical challenge execution follows this path:

1. Runner calls `challenge.Execute(ctx)`
2. Challenge template initializes its adapter
3. Challenge iterates over flow steps
4. Each step:
 - a. Reports progress via `BaseChallenge.ReportProgress()`
 - b. Dispatches to the adapter method
 - c. Creates an `AssertionResult` (passed/failed)
 - d. Records a `MetricValue` (duration, counts)
5. Challenge aggregates results
6. Returns `challenge.Result` with assertions, metrics, outputs

Integration Points

Challenge Framework

- Challenge templates embed `challenge.BaseChallenge` and implement `challenge.Challenge`.
- Results use `challenge.Result`, `challenge.AssertionResult`, `challenge.MetricValue`.

- Dependencies between challenges are expressed via `challenge.ID` slices passed to constructors.

Assertion Engine

- `UserFlowPlugin` registers evaluators with `assertion.DefaultEngine`.
- Evaluators follow the `assertion.Evaluator` function signature:
`func(def assertion.Definition, value any) (bool, string)`.

Containers Module

- `TestEnvironment` uses `containers/pkg/runtime`, `containers/pkg/compose`, `containers/pkg/health`, `containers/pkg/serviceregistry`, `containers/pkg/event`, and `containers/pkg/logging`.
- Auto-detects the container runtime (Podman-first).
- Health checking supports TCP, HTTP, and gRPC probes.

HTTP Client

- `HTTPAPIAdapter` wraps `pkg/httpclient.APIClient` for REST operations.
- Inherits JWT authentication, retry logic, and functional options from the `httpclient` package.
- Adds WebSocket support via `gorilla/websocket`.